

Q

What is the difference between DBMS and RDBMS?

DBMS

1. DBMS stores data as file.

2. No relationship between data.

3. Normalization is not present.

RDBMS

RDBMS stores data in **TABULAR** form.

Data is stored in the form of tables which are **RELATED** to each other. Eg: Foreign key relationship.

NORMALIZATION is present.

DBMS**RDBMS**

1. DBMS stores data as file.

RDBMS stores data in **TABULAR** form.

2. No relationship between data.

Data is stored in the form of tables which are **RELATED** to each other. Eg: Foreign key relationship.

3. Normalization is not present.

NORMALIZATION is present.

4. It deals with small quantity of data.

It deals with **LARGE** amount of data.

Q

What is the difference between Primary key and Unique key?

	Primary Key	Unique Key
1	Primary Key Can't Accept Null Values.	Unique Key Can Accept Only One Null Value
2	Creates Clustered Index	Creates Non-Clustered Index
3	Only One Primary key in a Table	More than One Unique Key in a Table.

Q

What are Triggers and types of triggers?

❖ Locations -> LocationHist

❖ Locations -> LocationHist

❖ Example of After(DML) Trigger

```
CREATE TRIGGER TR_UPD_Locations ON Locations
FOR UPDATE
NOT FOR REPLICATION
AS
BEGIN
    INSERT INTO LocationHist
    SELECT LocationID
           ,getdate()
    FROM inserted
END
```

Diagram annotations:

- Table Name: Points to `Locations`
- Trigger Name: Points to `TR_UPD_Locations`
- DML Event: Points to `FOR UPDATE`
- T-SQL block that runs against specified DML Event: Points to the `AS` block containing the `INSERT` and `SELECT` statements.

What are Triggers and types of triggers?

- ❖ Triggers are stored programs, which are **AUTOMATICALLY** executed or fired when some events (insert, delete and update) occur.
- ❖ Locations → LocationHist
- ❖ Example of After(DML) Trigger

```
CREATE TRIGGER TR_UPD_Locations ON Locations
FOR UPDATE
NOT FOR REPLICATION
AS
BEGIN
    INSERT INTO LocationHist
    SELECT LocationID
           ,getdate()
    FROM inserted
END
```

Diagram annotations:

- Table Name:** Points to `Locations` in `ON Locations`.
- DML Event:** Points to `FOR UPDATE`.
- Trigger Name:** Points to `TR_UPD_Locations`.
- T-SQL block that runs against specified DML Event:** Points to the entire trigger body (from `BEGIN` to `END`).

What are Triggers and types of triggers?

❖ Triggers are stored programs, which are **AUTOMATICALLY** executed or fired when some events (insert, delete and update) occur.

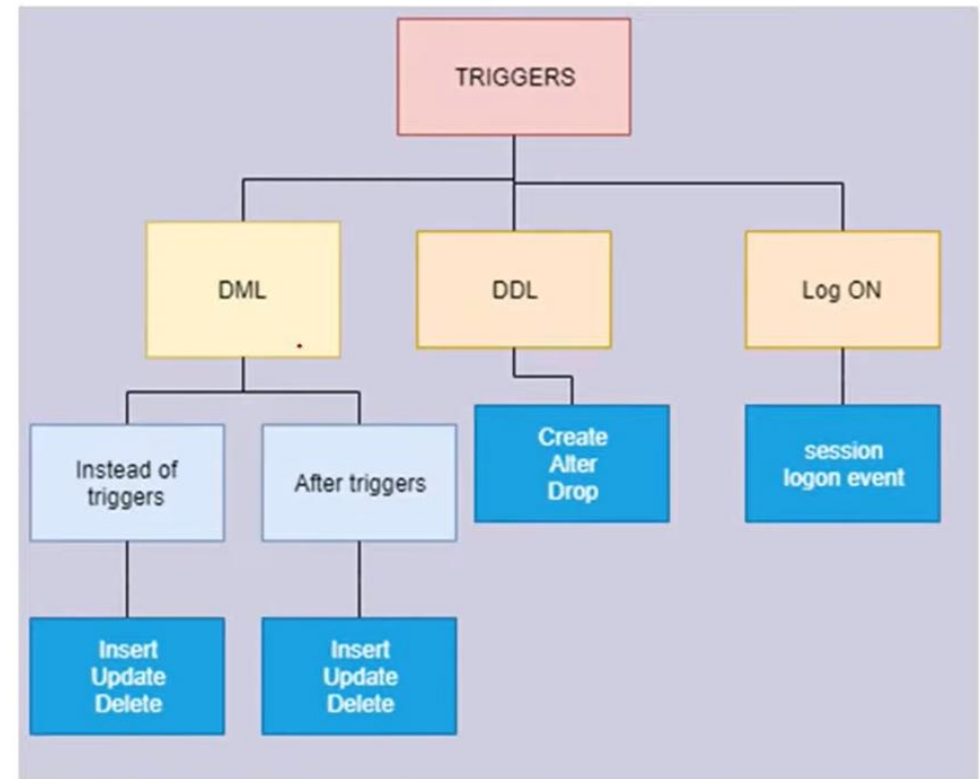
❖ Locations -> LocationHist

❖ Example of After(DML) Trigger

```
CREATE TRIGGER TR_UPD_Locations ON Locations
FOR UPDATE
NOT FOR REPLICATION
AS
BEGIN
    INSERT INTO LocationHist
    SELECT LocationID,
           ,getdate()
    FROM inserted
END
```

Annotations:

- Table Name:** Locations
- DML Event:** UPDATE
- Trigger Name:** TR_UPD_Locations
- T-SQL block that runs against specified DML Event:** The code block between BEGIN and END.



What are Triggers and types of triggers?

- ❖ Triggers are stored programs, which are **AUTOMATICALLY** executed or fired when some events (insert, delete and update) occur.

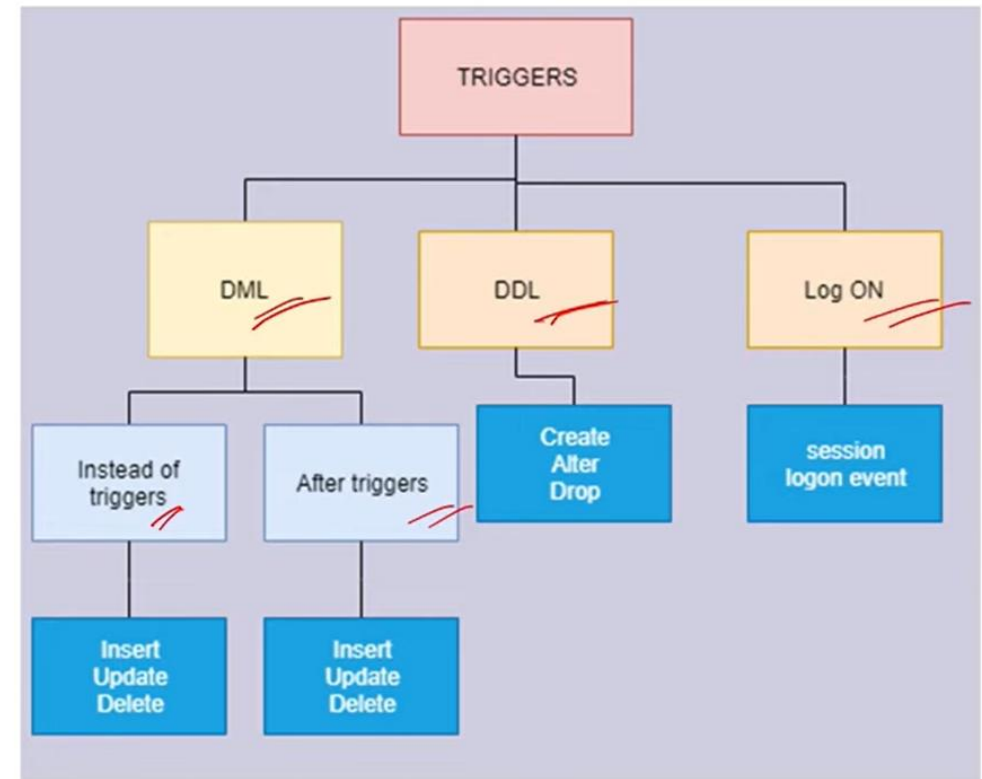
❖ Locations → LocationHist

- ❖ Example of After(DML) Trigger

```
CREATE TRIGGER TR_UPD_Locations ON Locations
FOR UPDATE
NOT FOR REPLICATION
AS
BEGIN
    INSERT INTO LocationHist
    SELECT LocationID
           ,getdate()
    FROM inserted
END
```

Annotations:

- Table Name:** Locations
- DML Event:** UPDATE
- Trigger Name:** TR_UPD_Locations
- T-SQL block that runs against specified DML Event:** The code block between BEGIN and END.



- ❖ In after trigger, update on the table executed first and then trigger will run.

- ❖ Triggers are stored programs, which are **AUTOMATICALLY** executed or fired when some events (insert, delete and update) occur.

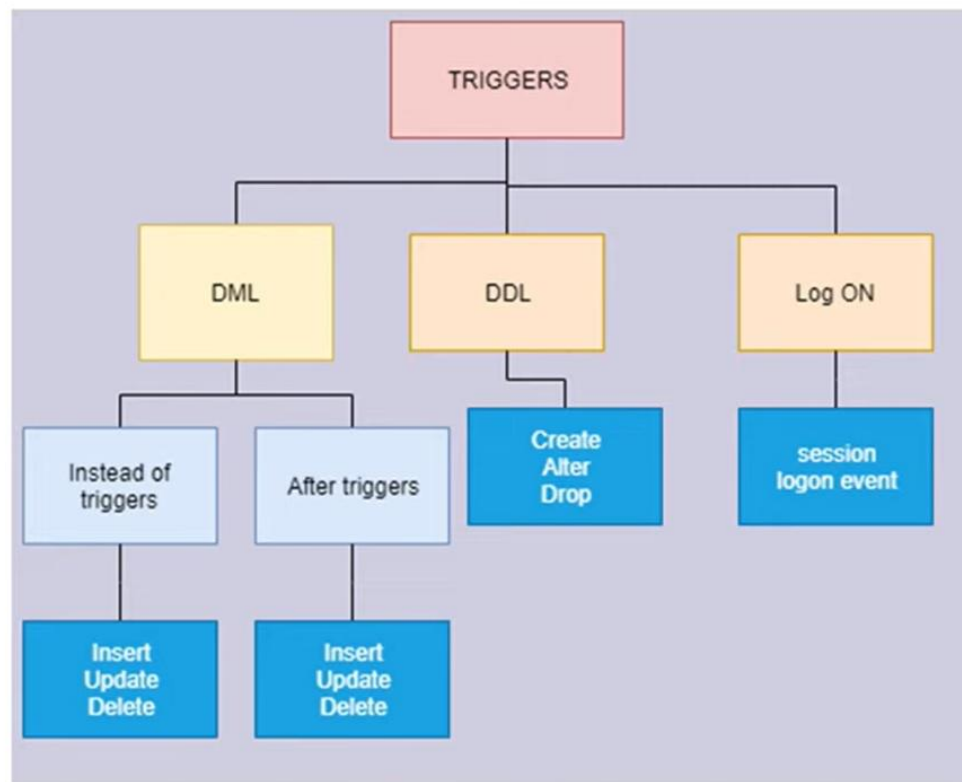
- ❖ Example of Instead of(DML) Trigger

```

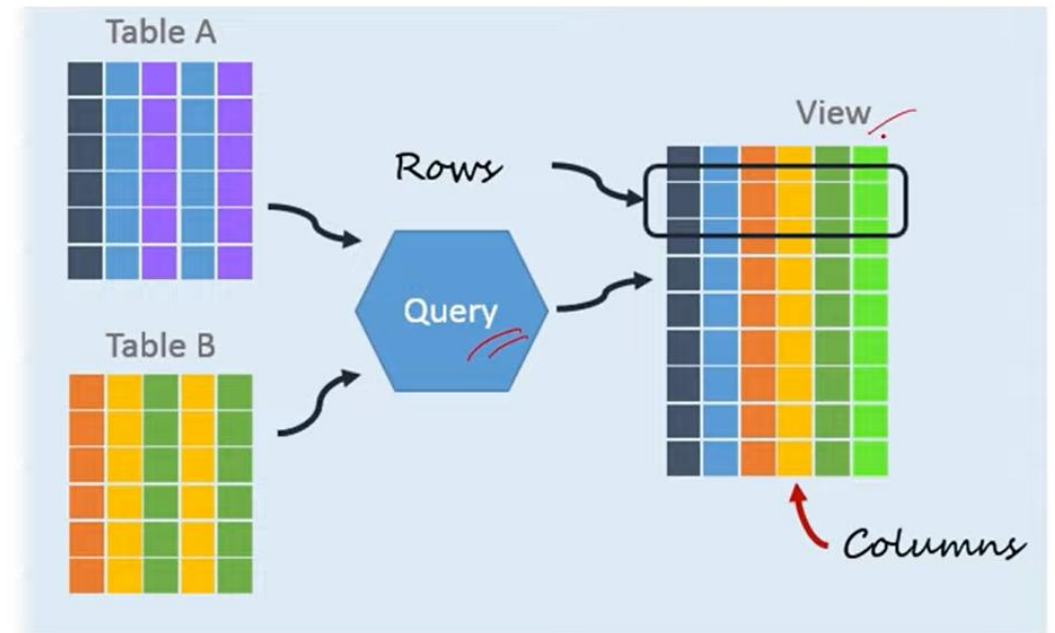
1
2 CREATE TRIGGER [dbo].[TRG_VM_EMPDETAILS]
3 ON [dbo].[VW_EMPDETAILS]
4 INSTEAD OF INSERT
5 AS
6 BEGIN
7     -- LOGIC HERE
8
9 END
10
11
12 This trigger is only for INSERT
13

```

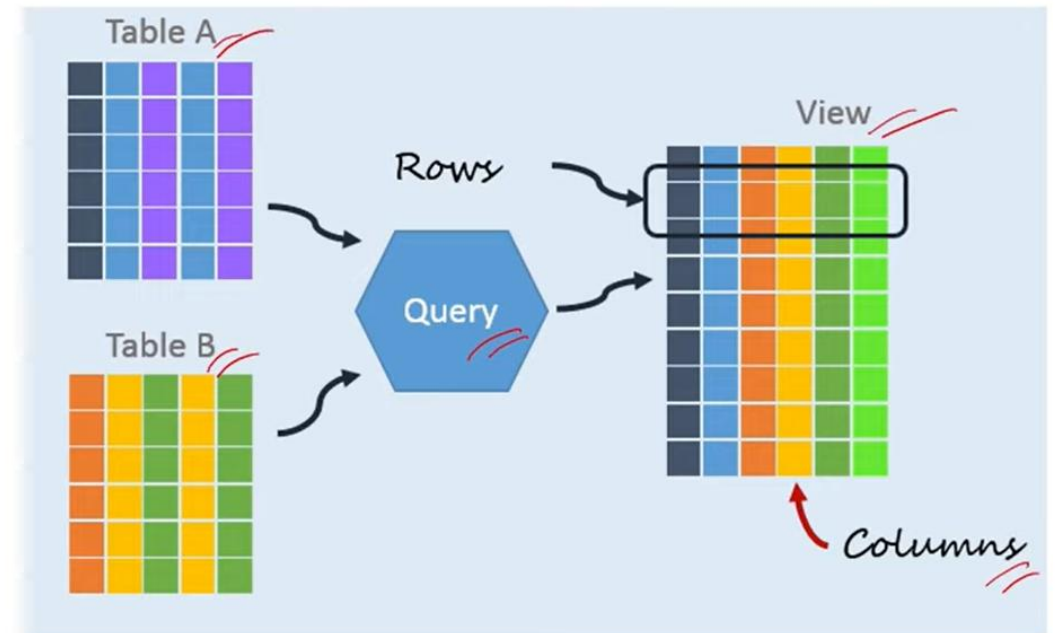
Trigger Name
 View Name
 INSTEAD OF Trigger



- ❖ In INSTEAD OF trigger, you can skip an INSERT, DELETE or UPDATE statement to a table and execute other statements defined in the trigger instead.



- ❖ A view is a **VIRTUAL table** which consists of a subset of data contained in a table or more than one table.



- ❖ Remember, in a view query is stored but the data is never stored like a table.

- ❖ A view is a VIRTUAL table which consists of a subset of data contained in a table or more than one table.

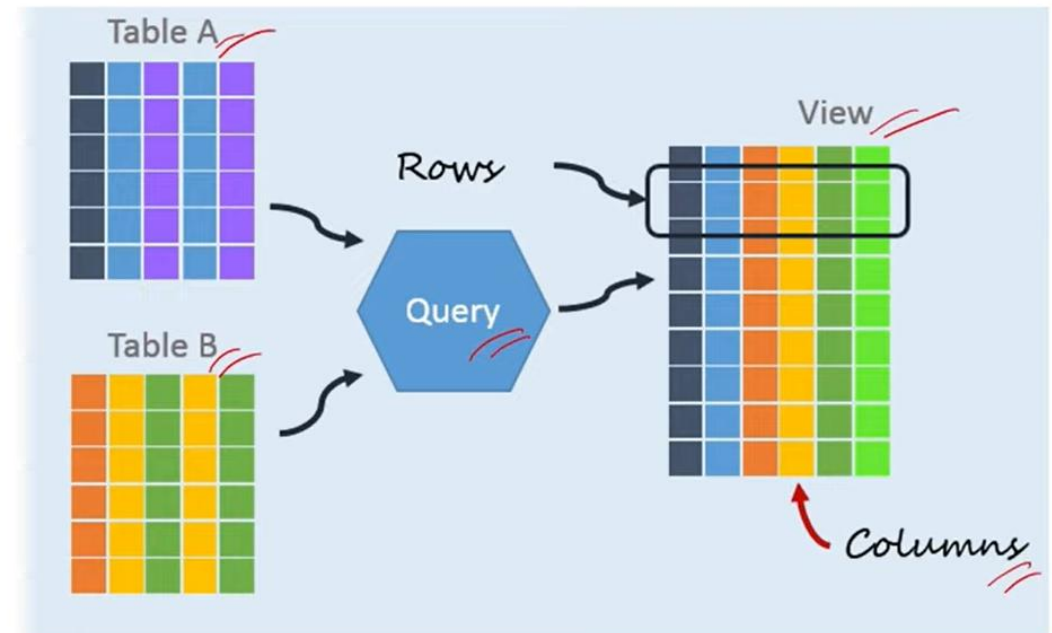
```
CREATE VIEW [India-Customers] AS
```

```
SELECT CustomerName, ContactName
```

```
FROM Customers
```

```
WHERE Country = 'India';
```

- ❖ Views are not stored in memory like tables then why to use views.



- ❖ Remember, in a view query is stored but the data is never stored like a table.

- ❖ A view is a VIRTUAL table which consists of a subset of data contained in a table or more than one table.

```
CREATE VIEW [India-Customers] AS
```

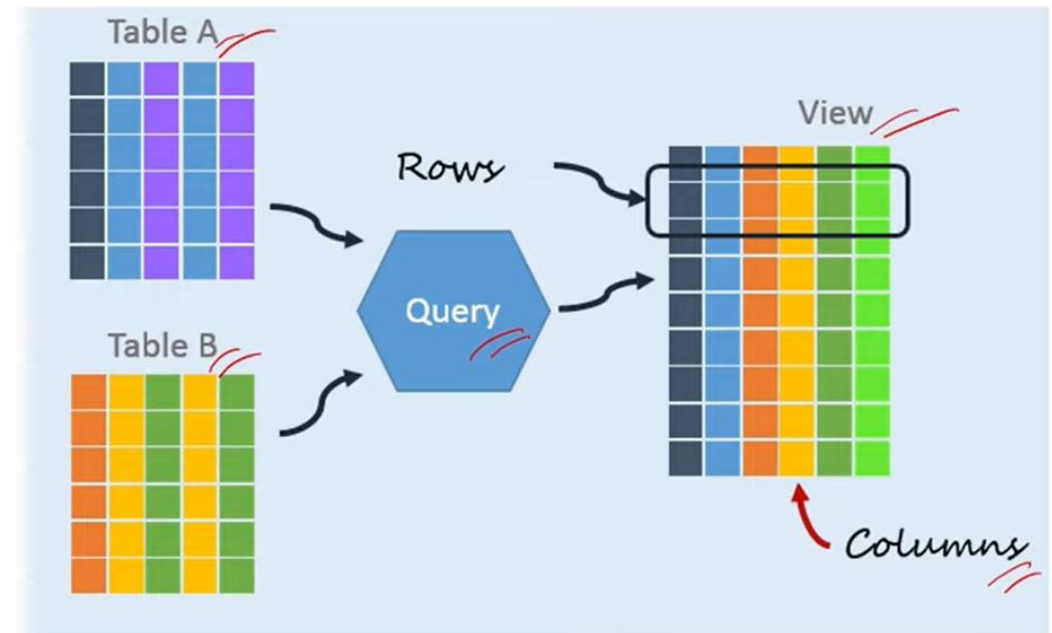
```
SELECT CustomerName, ContactName
```

```
FROM Customers
```

```
WHERE Country = 'India';
```

- ❖ Views are not stored in memory like tables then why to use views.

1. Indexed Views to improve the performance.



- ❖ Remember, in a view query is stored but the data is never stored like a table.

- ❖ A view is a VIRTUAL table which consists of a subset of data contained in a table or more than one table.

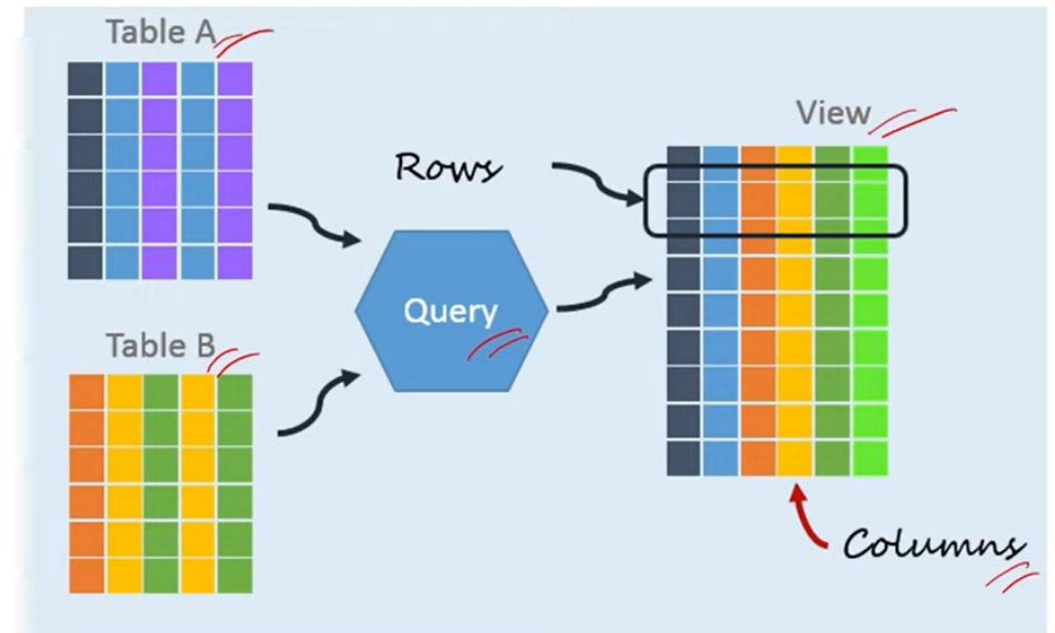
```
CREATE VIEW [India-Customers] AS
```

```
SELECT CustomerName, ContactName
```

```
FROM Customers
```

```
WHERE Country = 'India';
```

- ❖ Views are not stored in memory like tables then why to use views.
1. Indexed Views to improve the performance.
 2. Extra security – DBA can hide the actual table names and expose views for Read operations.



- ❖ Remember, in a view query is stored but the data is never stored like a table.


```
SELECT LastName, FirstName
```

```
FROM Employees
```

```
WHERE OfficeCode IN (SELECT OfficeCode  
 FROM Offices  
 WHERE Country = 'USA');
```

Outer Query

Inner Query

- ❖ Auto-increment allows a unique number to be **generated automatically** when a new record is inserted into a table.
- ❖ Mostly it is the primary key only.

```
CREATE TABLE Persons (  
    Personid int IDENTITY(1,1) PRIMARY KEY,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int  
);
```

Handwritten notes: 1, 2, 3, ... above the IDENTITY function; red underlines under Personid, IDENTITY, and 1,1.

DELETE

1. It is a DML.

2. It is used to delete one or all rows from the table based on where condition, but it will not delete schema.

```
DELETE FROM Employees  
WHERE Emp_Id = 7;
```

TRUNCATE

1. It is a DDL.

2. It is used to delete all rows from the table, but it will not delete schema.

```
TRUNCATE TABLE Employees;
```

DROP

1. It is a DDL.

2. It is used to delete all rows from the table with structure/schema.

```
DROP TABLE Employees;
```

DELETE**TRUNCATE****DROP**

1. It is a DML.

1. It is a DDL.

1. It is a DDL.

2. It is used to delete one or all rows from the table based on where condition, but it will not delete schema.

2. It is used to delete all rows from the table, but it will not delete schema.

2. It is used to delete all rows from the table with structure/schema.

3. It can be rollback.

3. It can not be rollback.

3. It can not be rollback.

```
DELETE FROM Employees
WHERE Emp_Id = 7;
```

```
TRUNCATE TABLE Employees;
```

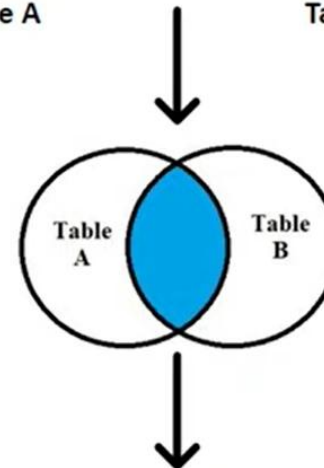
```
DROP TABLE Employees;
```

Student ID	Name
1001	A
1002	B
1003	C
1004	D

Student ID	Department
1004	Mathematics
1005	Mathematics
1006	History
1007	Physics
1008	Computer Science

Table A

Table B



Student ID	Name	Department
1004	D	Mathematics

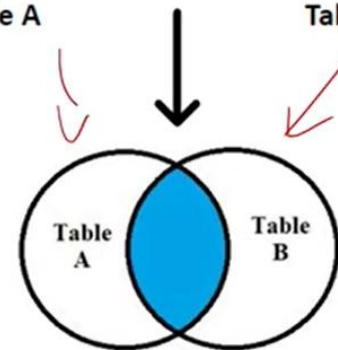
- ❖ A join clause is used to COMBINE rows from two or more tables, based on a related column between them.

Student ID	Name
1001	A
1002	B
1003	C
1004	D

Student ID	Department
1004	Mathematics
1005	Mathematics
1006	History
1007	Physics
1008	Computer Science

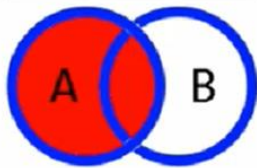
Table A

Table B

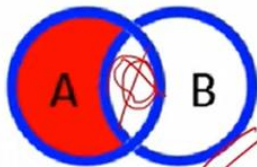


Student ID	Name	Department
1004	D	Mathematics

LEFT OUTER JOIN

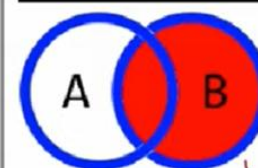


```
SELECT *
FROM TableA a
LEFT JOIN TableB b
  ON a.KEY = b.KEY
```

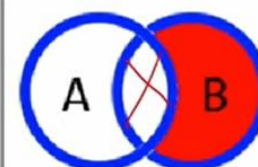


```
SELECT *
FROM TableA a
LEFT JOIN TableB b
  ON a.KEY = b.KEY
WHERE b.KEY IS NULL
```

RIGHT OUTER JOIN

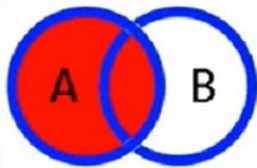


```
SELECT *
FROM TableA a
RIGHT JOIN TableB b
  ON a.KEY = b.KEY
```

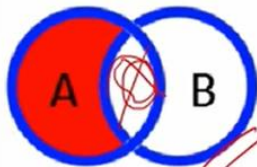


```
SELECT *
FROM TableA a
RIGHT JOIN TableB b
  ON a.KEY = b.KEY
WHERE a.KEY IS NULL
```

LEFT OUTER JOIN

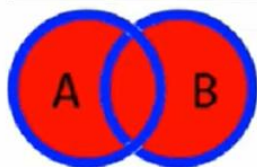


```
SELECT *
FROM TableA a
LEFT JOIN TableB b
ON a.KEY = b.KEY
```

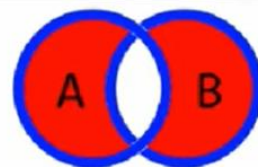


```
SELECT *
FROM TableA a
LEFT JOIN TableB b
ON a.KEY = b.KEY
WHERE b.KEY IS NULL
```

FULL OUTER JOIN

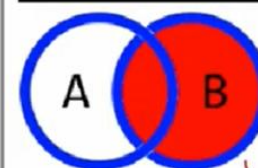


```
SELECT *
FROM TableA a
FULL OUTER JOIN TableB b
ON a.KEY = b.KEY
```

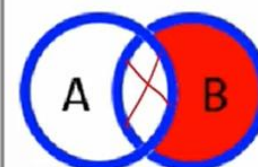


```
SELECT *
FROM TableA a
FULL OUTER JOIN TableB b
ON a.KEY = b.KEY
WHERE a.KEY IS NULL
OR b.KEY IS NULL
```

RIGHT OUTER JOIN

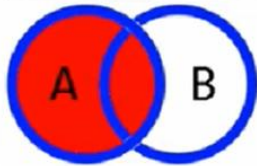


```
SELECT *
FROM TableA a
RIGHT JOIN TableB b
ON a.KEY = b.KEY
```

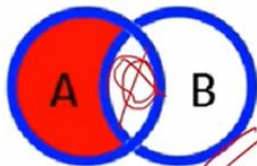


```
SELECT *
FROM TableA a
RIGHT JOIN TableB b
ON a.KEY = b.KEY
WHERE a.KEY IS NULL
```


LEFT OUTER JOIN

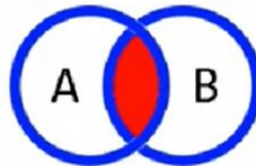


```
SELECT *
FROM TableA a
LEFT JOIN TableB b
ON a.KEY = b.KEY
```



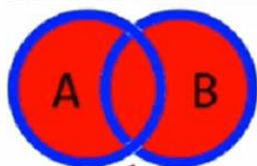
```
SELECT *
FROM TableA a
LEFT JOIN TableB b
ON a.KEY = b.KEY
WHERE b.KEY IS NULL
```

INNER JOIN

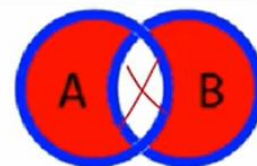


```
SELECT *
FROM TableA a
INNER JOIN TableB b
ON a.KEY = b.KEY
```

FULL OUTER JOIN

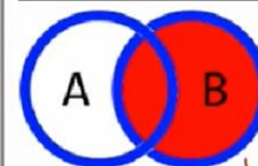


```
SELECT *
FROM TableA a
FULL OUTER JOIN TableB b
ON a.KEY = b.KEY
```

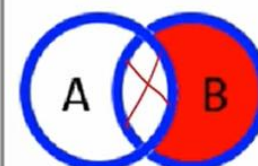


```
SELECT *
FROM TableA a
FULL OUTER JOIN TableB b
ON a.KEY = b.KEY
WHERE a.KEY IS NULL
OR b.KEY IS NULL
```

RIGHT OUTER JOIN



```
SELECT *
FROM TableA a
RIGHT JOIN TableB b
ON a.KEY = b.KEY
```



```
SELECT *
FROM TableA a
RIGHT JOIN TableB b
ON a.KEY = b.KEY
WHERE a.KEY IS NULL
```


❖ When to use Self Join??

employees	
* employee_id	
first_name	
email	
phone_number	
hire_date	
job_id	
salary	
manager_id	123
department_id	

This manager id is
employee id of the
manager of an
employee

❖ Now your task is to get the employees
name with their manager names??

SELECT

e.first_name AS employee,
m.first_name AS manager

FROM

employees e LEFT JOIN
employees m

ON m.employee_id = e.manager_id

ORDER BY manager;

employee	manager
▶ Steven King	NULL
Bruce Ernst	Alexander Hunold
David Austin	Alexander Hunold
Valli Pataballa	Alexander Hunold
Diana Lorentz	Alexander Hunold
Alexander Khoo	Den Raphaely

- ❖ A self join is **a join of a table to itself**.
- ❖ When to use Self Join??

employees	
* employee_id	
first_name	
email	
phone_number	
hire_date	
job_id	
salary	
manager_id	123
department_id	

This manager id is
employee id of the
manager of an
employee

- ❖ Now your task is to get the employees name with their manager names??

SELECT

e.first_name AS employee,
m.first_name AS manager

FROM

employees e LEFT JOIN
employees m

ON m.employee_id = e.manager_id

ORDER BY manager;

employee	manager
Steven King	NULL
Bruce Ernst	Alexander Hunold
David Austin	Alexander Hunold
Valli Pataballa	Alexander Hunold
Diana Lorentz	Alexander Hunold
Alexander Khoo	Den Raphaely



CHAPTER 3 - INDEXES

Q14. What are Indexes in SQL Server?

Q15. What is Clustered index?

Q16. What is Non-Clustered index?

Q17. What is the difference between Clustered and Non-Clustered index? **V. IMP.**

Q18. How to create Clustered and Non-Clustered index in a table?

Q19. In which column you will apply the indexing to optimize this query?

- ❖ SQL Indexes are used in relational databases to retrieve data **VERY FAST**.

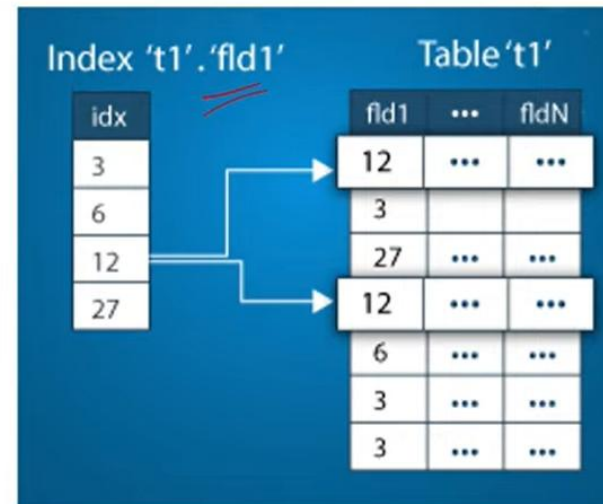


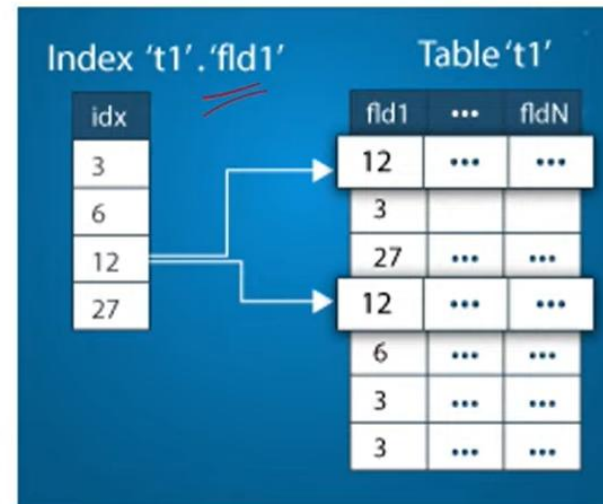
Table of Contents

Acknowledgments	ix
Introduction	xi

Part I Envision the Possibilities

1 Welcome to Office 2010	3
Features that Fit Your Work Style	3
Changes in Office 2010	4
Let Your Ideas Soar	5
Collaborate Easily and Naturally	5
Work Anywhere—and Everywhere	6
Exploring the Ribbon	7

- ❖ SQL Indexes are used in relational databases to retrieve data **VERY FAST**.



- ❖ They are similar to indexes at the start of the BOOKS, which purpose is to find a topic quickly.

Table of Contents

Acknowledgments	ix
Introduction	xi

Part I Envision the Possibilities

1 Welcome to Office 2010	3
Features that Fit Your Work Style	3
Changes in Office 2010	4
Let Your Ideas Soar	5
Collaborate Easily and Naturally	5
Work Anywhere—and Everywhere	6
Exploring the Ribbon	7

- ❖ A clustered index defines the order in which data is **physically** stored in a table.

Dictionary



- ❖ A clustered index defines the order in which data is **physically** stored in a table.

Dictionary



- ❖ A clustered index defines the order in which data is **physically** stored in a table.
- ❖ Table data can be sorted in only way, therefore, there can be only one clustered index per table.
- ❖ In SQL Server, if you set a primary key on a column then it will automatically create a clustered index on that particular column.

Dictionary



- ❖ A non-clustered index is stored at one place and table data is stored in another place.
- ❖ A table can have multiple non-clustered index in a table.

Index

A

accordion, layouts
 about 128
 movie form, adding 131
 nesting, in tab 128, 129
 toolbar, adding 129-131
 adapters, Ext
 about 18
 using 18, 20
 Adobe AIR 285
 Adobe Integrated Run time. *See* Adobe AIR
 AJAX 12
 Asynchronous JavaScript and XML.
 See AJAX

B

built-in features, Ext
 client-side sorting 86
 column, reordering 86, 87
 columns, hidden 86

lookup data stores, creating 83
 two columns, combining 84
 classes 254
 ComboBox, form
 about 47
 database-driven 47-50
 component config 59
 config object
 about 28, 29
 new way 28, 29
 old way 28
 tips 26, 29
 content, loading on menu item click 68, 69
 custom class, creating 256-259
 custom component, creating 264-266
 custom events, creating 262-264

D

data, filtering
 about 238
 remote, filtering 238-244

Book Index

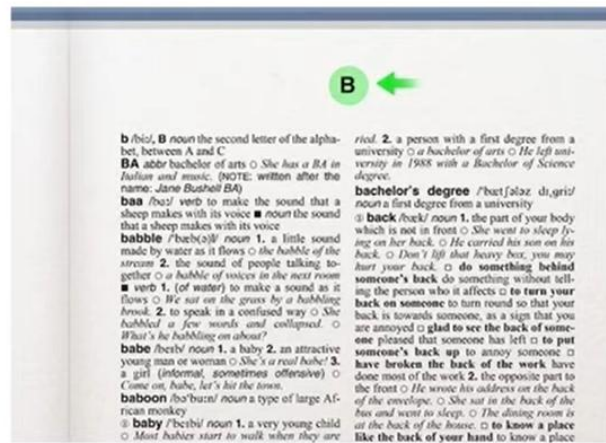
Table of Contents

Acknowledgments	ix
Introduction	xi

Part I Envision the Possibilities

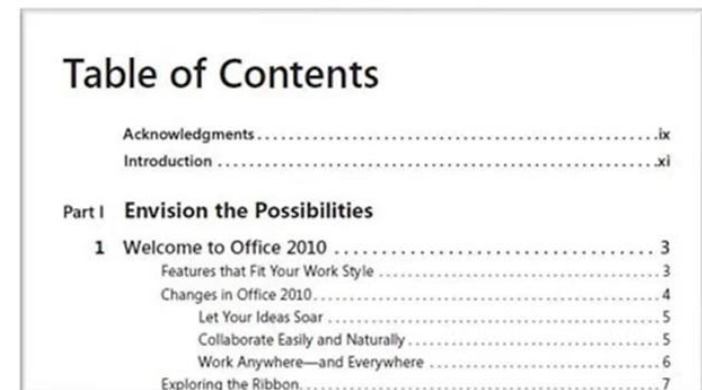
1 Welcome to Office 2010	3
Features that Fit Your Work Style	3
Changes in Office 2010	4
Let Your Ideas Soar	5
Collaborate Easily and Naturally	5
Work Anywhere—and Everywhere	6
Exploring the Ribbon	7

A non-clustered index is stored at one place and table data is stored in another place. For example, Book Index.



1	Welcome to Office 2010	3
	Features that Fit Your Work Style	3
	Changes in Office 2010	4
	Let Your Ideas Soar	5
	Collaborate Easily and Naturally	5
	Work Anywhere—and Everywhere	6
	Exploring the Ribbon	7

- Non-clustered index is slower.



❖ CLUSTERED INDEX

```
CREATE CLUSTERED INDEX <index_name>  
ON <table_name>(<column_name> ASC/DESC)
```

❖ CLUSTERED INDEX

```
CREATE CLUSTERED INDEX <index_name>  
ON <table_name>(<column_name> ASC/DESC)
```

❖ NON-CLUSTERED INDEX

```
CREATE NONCLUSTERED INDEX <index_name>  
ON <table_name>(<column_name> ASC/DESC)
```


❖ CLUSTERED INDEX

```
CREATE CLUSTERED INDEX <index_name>  
ON <table_name>(<column_name> ASC/DESC)
```

❖ NON-CLUSTERED INDEX

```
CREATE NONCLUSTERED INDEX <index_name>  
ON <table_name>(<column_name> ASC/DESC)
```

- ❖ When you create a PRIMARY KEY constraint, a clustered index on the column or columns is automatically created.

Q

In which column you will apply the indexing to optimize this query.
“select id, class from student where name=“happy””?

```
select id, class from student where name="happy"
```

Q

In which column you will apply the indexing to optimize this query.
“select id, class from student where name=“happy””?

select id, class from student where name=“happy”

- ❖ The column **after WHERE** condition, which is “NAME” here.



CHAPTER 4 - STORED PROCEDURE, FUNCTIONS & OTHERS

Q20. What is the difference between Stored Procedure and Functions? **V. IMP.**

Q21. What is a Cursor? Why to avoid them?

Q22. What is CTE in SQL Server?

Q23. How to optimize a Stored Procedure or SQL Query?

Q24. How to get the Nth highest salary of an employee?

Q25. What are ACID properties?



What is the difference between Stored Procedure and Functions (at least 3)?

V. IMP.



Stored Procedure

1. SP may or may not **return** a value

Function

Function must return a value

```
CREATE PROCEDURE proc_name  
(@Ename varchar(50),  
@Eid int output)  
AS  
BEGIN
```

```
INSERT INTO Employee (EmpName)  
VALUES (@Ename)  
SELECT @Eid= SCOPE_IDENTITY()
```

```
END
```

```
CREATE FUNCTION function_name  
(parameters) --only input parameter  
RETURNS data_type AS  
BEGIN
```

```
-- SQL statements  
RETURN value
```

```
END;
```



Stored Procedure

Function

- | | |
|---|----------------------------------|
| 1. SP may or may not return a value | Function must return a value |
| 2. Can have input/ output parameters | Only has input parameters |
| 3. We can call function inside SP | Cannot call SP inside a function |

```
CREATE PROCEDURE proc_name
```

```
(@Ename varchar(50),
```

```
@Eid int output)
```

```
AS
```

```
BEGIN
```

```

-- function
INSERT INTO Employee (EmpName)
VALUES (@Ename)
SELECT @Eid= SCOPE_IDENTITY()
```

```
END
```

```
CREATE FUNCTION function_name
```

```
(parameters) --only input parameter
```

```
RETURNS data_type AS
```

```
BEGIN
```

```

-- SQL statements
```

```
RETURN value
```

```
END;
```


Stored Procedure

Function

1. SP may or may not **return** a value

Function must return a value

2. Can have input/**output** parameters

Only has input parameters

3. We can **call** function inside SP

Cannot call SP inside a function

4. We cannot use SP in **SQL statements** like SELECT, INSERT, UPDATE, DELETE, MERGE, etc.We can use them with function.
*SELECT *, dbo.fnCountry(city.long) FROM city;*

5. We can use try-catch exception handling in SP

We can not use try-catch in functions

6. We can use transactions inside SP.

We can not use transactions inside functions.

CREATE PROCEDURE *proc_name*(*@Ename* *varchar*(50),*@Eid* *int* output)

AS

BEGIN

-----function

 INSERT INTO *Employee* (*EmpName*)
VALUES (*@Ename*)
SELECT *@Eid*=*SCOPE_IDENTITY()*

END

CREATE FUNCTION *function_name*(*parameters*) --only input parameter*RETURNS* *data_type* AS

BEGIN

SP X

 -- *SQL statements*
RETURN *value*

END;

What is a Cursor? Why to avoid them?

❖ A database Cursor is a control which enables traversal/ iteration over the rows or records in the table.

❖ 5 step process:

1. Declare
2. Open
3. Fetch using while loop
4. Close
5. Deallocate

```

DECLARE
    @product_name VARCHAR(MAX),
    @list_price DECIMAL;

DECLARE cursor_product CURSOR
FOR SELECT
    product_name,
    list_price
FROM
    production.products;

OPEN cursor_product;

FETCH NEXT FROM cursor_product INTO
    @product_name,
    @list_price;

WHILE @@FETCH_STATUS = 0
BEGIN
    PRINT @product_name + CAST(@list_price AS varchar);
    FETCH NEXT FROM cursor_product INTO
        @product_name,
        @list_price;
END;

CLOSE cursor_product;

DEALLOCATE cursor_product;
  
```

What is a Cursor? Why to avoid them?

❖ A database Cursor is a control which enables traversal/ iteration over the rows or records in the table.

❖ 5 step process:

1. Declare
2. Open
3. Fetch using while loop
4. Close
5. Deallocate

❖ LIMITATION

A cursor is a MEMORY resident set of pointers - meaning it occupies lots of memory from your system which is not good for performance.

```

DECLARE
    @product_name VARCHAR(MAX),
    @list_price DECIMAL;

DECLARE cursor_product CURSOR
FOR SELECT
    product_name,
    list_price
FROM
    production.products;

OPEN cursor_product;

FETCH NEXT FROM cursor_product INTO
    @product_name,
    @list_price;

WHILE @@FETCH_STATUS = 0
BEGIN
    PRINT @product_name + CAST(@list_price AS varchar);
    FETCH NEXT FROM cursor_product INTO
        @product_name,
        @list_price;
END;

CLOSE cursor_product;

DEALLOCATE cursor_product;
  
```

```
with engineers as (  
  select *  
  from employees  
  where dept='Engineering'  
)
```

Annotations:

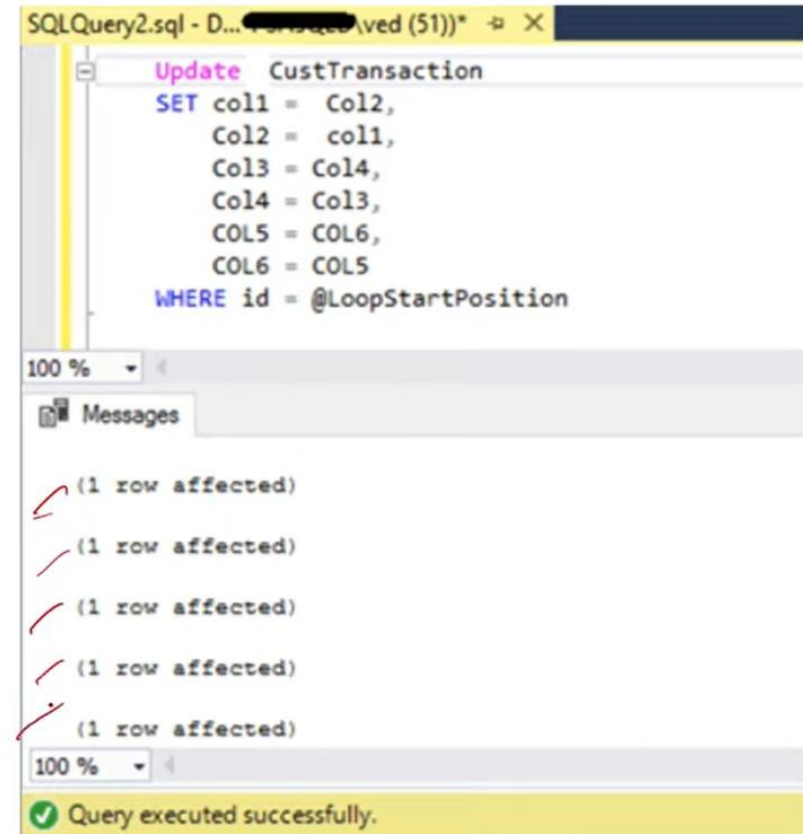
- WITH
- CTE name (pointing to `engineers`)
- CTE Body (pointing to the select statement inside the parentheses)

```
select *  
from engineers  
where ...
```

Annotation:

- CTE Usage (pointing to `engineers`)

1. Use SET NOCOUNT ON



```
SQLQuery2.sql - D:\... (ved (51))* X
```

```
Update CustTransaction
SET col1 = Col2,
    Col2 = col1,
    Col3 = Col4,
    Col4 = Col3,
    COL5 = COL6,
    COL6 = COL5
WHERE id = @LoopStartPosition
```

100 %

Messages

(1 row affected)

(1 row affected)

(1 row affected)

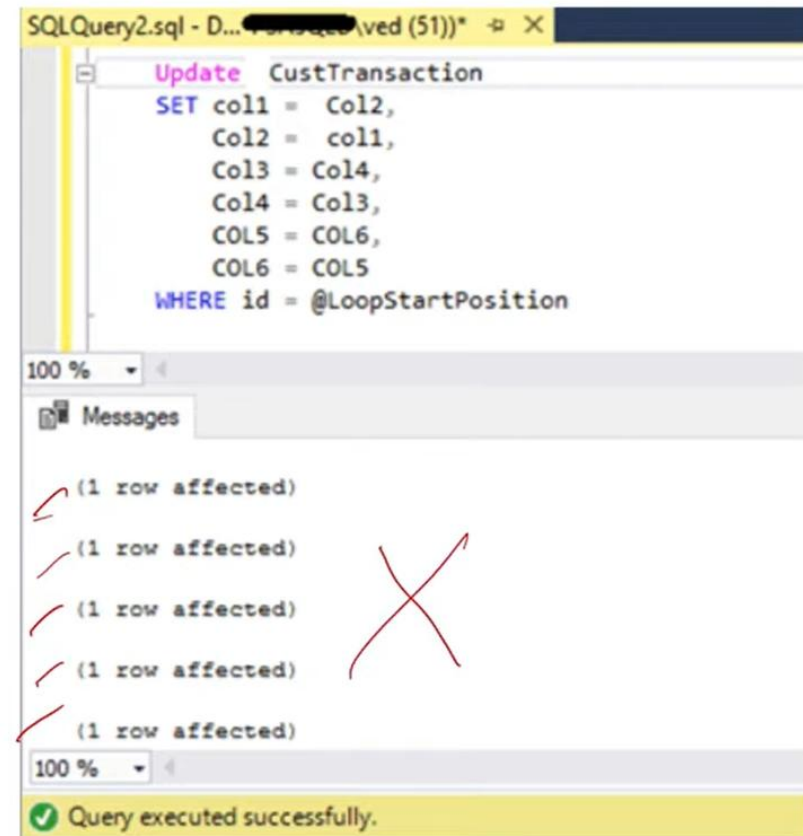
(1 row affected)

(1 row affected)

100 %

✓ Query executed successfully.

1. Use SET NOCOUNT ON
2. Specify column names instead of using * .
SELECT col1, col2 FROM table1

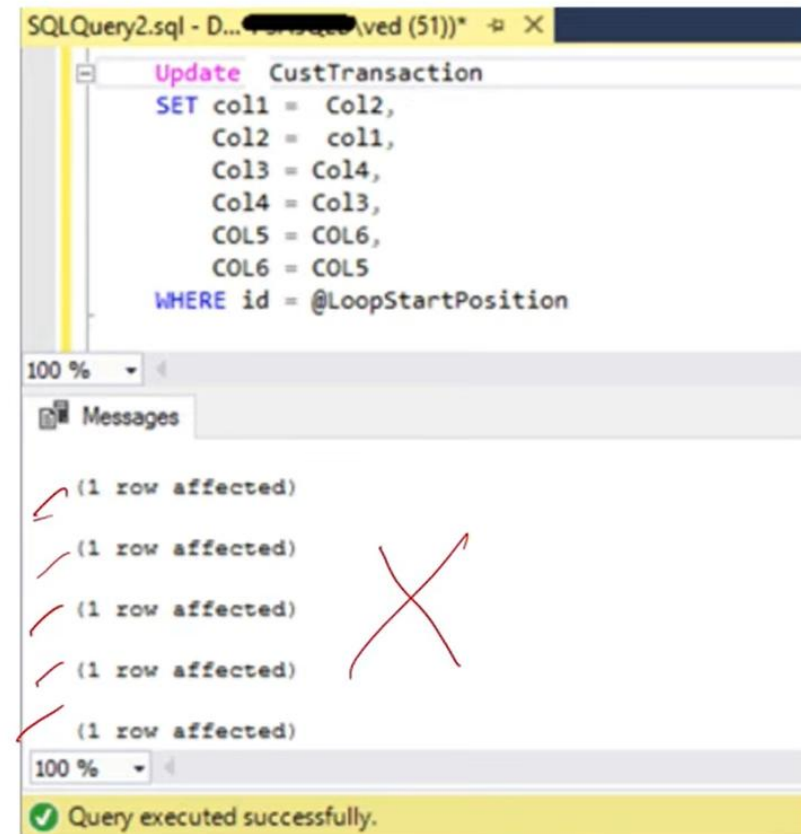


The screenshot shows a SQL query window titled "SQLQuery2.sql - D:\... (ved (51))". The query is an UPDATE statement for a table named "CustTransaction". The query text is:

```
Update CustTransaction
SET col1 = Col2,
    Col2 = col1,
    Col3 = Col4,
    Col4 = Col3,
    COL5 = COL6,
    COL6 = COL5
WHERE id = @LoopStartPosition
```

Below the query window, the "Messages" pane shows the execution results. It displays five messages, each stating "(1 row affected)". A large red 'X' is drawn over these messages. At the bottom of the Messages pane, a green checkmark icon is followed by the text "Query executed successfully."

1. Use SET NOCOUNT ON
2. Specify column names instead of using * .
SELECT col1, col2 FROM table1
3. Use schema name before objects or tablename.
SELECT EmpID, Name FROM dbo.Employee
4. Do not use DYNAMIC QUERIES. They are vulnerable to SQL Injections.
5. Use EXISTS () instead of COUNT ().
Incorrect: *SELECT Count(1) FROM dbo.Employee* ✗
Correct: *IF(EXISTS (SELECT 1 FROM db.Employees))* ✓

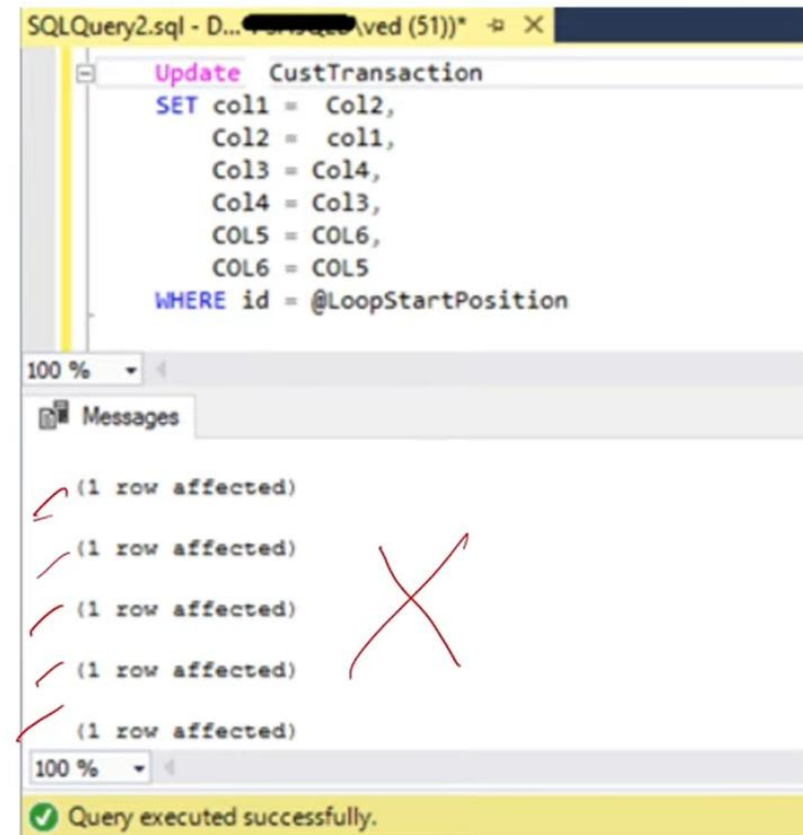


```

SQLQuery2.sql - D:\... (ved (51)) * X
Update CustTransaction
SET col1 = Col2,
    Col2 = col1,
    Col3 = Col4,
    Col4 = Col3,
    COL5 = COL6,
    COL6 = COL5
WHERE id = @LoopStartPosition
100 %
Messages
(1 row affected)
(1 row affected)
(1 row affected)
(1 row affected)
(1 row affected)
100 %
Query executed successfully.

```


1. Use SET NOCOUNT ON
2. Specify column names instead of using *.
`SELECT col1, col2 FROM table1`
3. Use schema name before objects or tablenames.
`SELECT EmpID, Name FROM dbo.Employee`
4. Do not use DYNAMIC QUERIES. They are vulnerable to SQL Injections.
5. Use EXISTS () instead of COUNT ().
 Incorrect: `SELECT Count(1) FROM dbo.Employee`
 Correct: `IF( EXISTS (SELECT 1 FROM db.Employees))`
6. Use TRANSACTION when required only



```

SQLQuery2.sql - D:\... (ved (51))* X
Update CustTransaction
SET col1 = Col2,
    Col2 = col1,
    Col3 = Col4,
    Col4 = Col3,
    COL5 = COL6,
    COL6 = COL5
WHERE id = @LoopStartPosition
  
```

100 %

Messages

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

100 %

✓ Query executed successfully.

1. The logic is first select TOP 3 salaries in descending order.

2. Put the result in "Result" and then do order by asc

**SELECT SALARY
FROM (**

**SELECT DISTINCT TOP N SALARY
FROM tbl_Employees
ORDER BY SALARY DESC**

**SELECT DISTINCT TOP N SALARY
FROM tbl_Employees
ORDER BY SALARY DESC**

**) RESULT
ORDER BY SALARY**

Salary
5000
10000
6000
4000
2000
7000

10000
7000
6000

Result
6000
7000
10000

1. The logic is first select TOP 3 salaries in descending order.

2. Put the result in "Result" and then do order by asc

3. Select top 1 salary from result set

**SELECT SALARY
FROM (**

**SELECT TOP 1 SALARY
FROM (**

**SELECT DISTINCT TOP N SALARY
FROM tbl_Employees
ORDER BY SALARY DESC**

**SELECT DISTINCT TOP N SALARY
FROM tbl_Employees
ORDER BY SALARY DESC**

**SELECT DISTINCT TOP N SALARY
FROM tbl_Employees
ORDER BY SALARY DESC**

**) RESULT
ORDER BY SALARY**

**) RESULT
ORDER BY SALARY**

Salary
5000
10000
6000
4000
2000
7000

10000
7000
6000

Result
6000
7000
10000

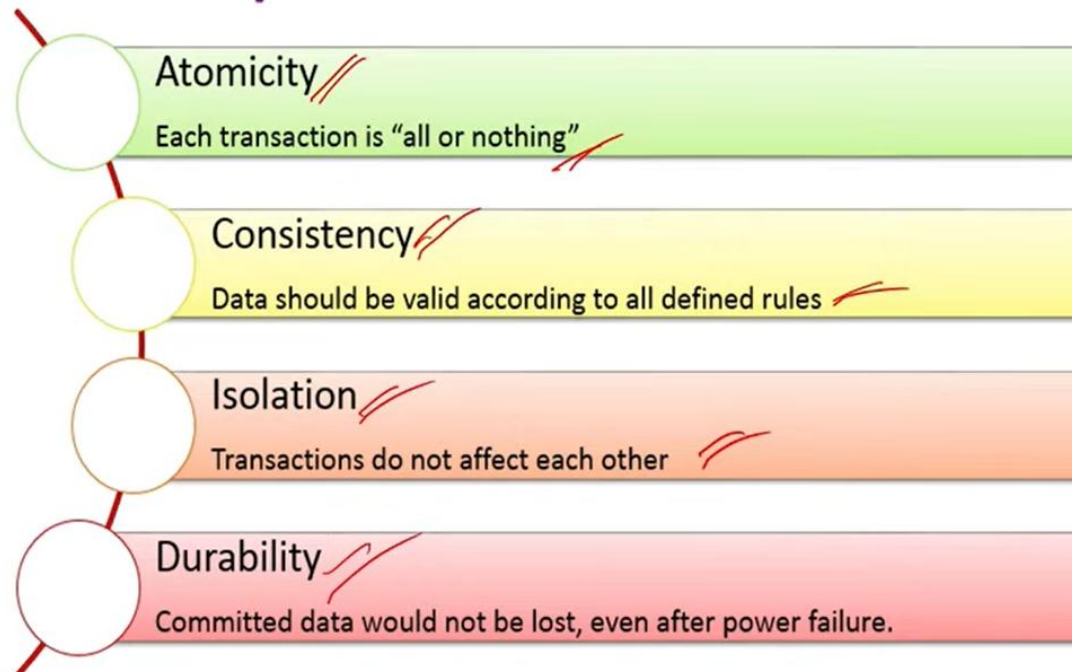
Result
6000

What are ACID properties?

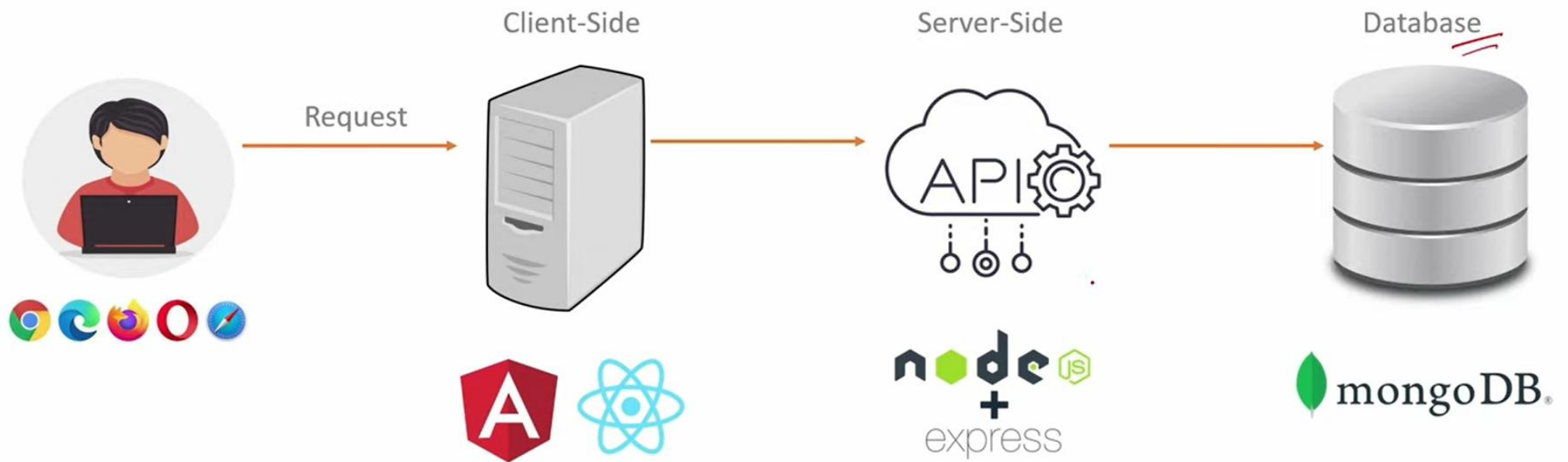
- ❖ ACID properties are used when you are handling **transactions** in SQL.
- ❖ For example, multiple inserts are coming at same point of time.

What are ACID properties?

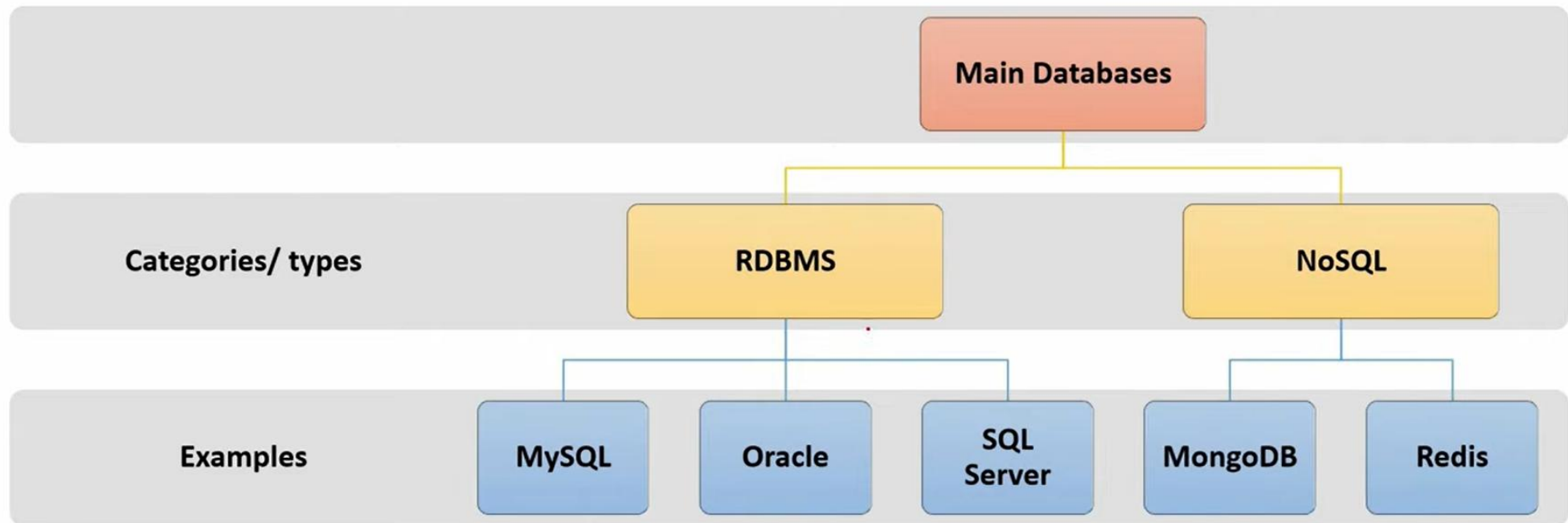
- ❖ ACID properties are used when you are handling **transactions** in SQL.
- ❖ For example, multiple inserts are coming at same point of time.



Q. What is MongoDB? **V. IMP.**



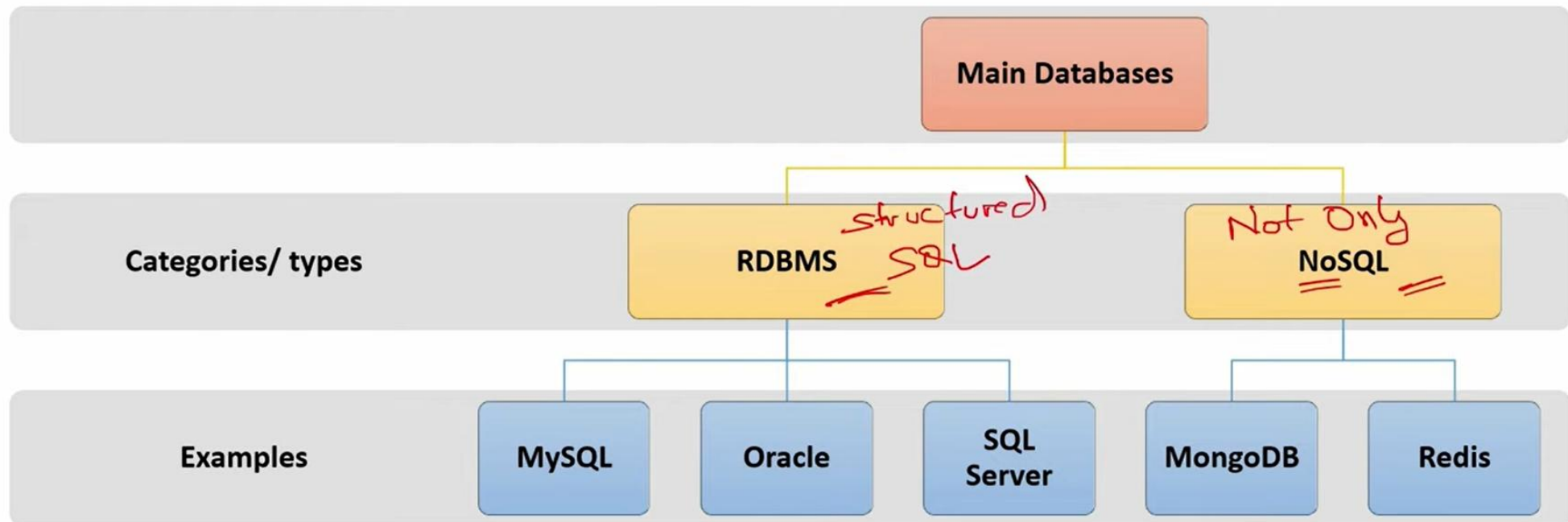
Q. What is **NoSQL**?



Q. What is **NoSQL**?



- ❖ NoSQL(Not Only SQL) databases are designed to handle structured, semi-structured, and unstructured data.



Q. What is the difference between NoSQL & RDBMS? **V. IMP.**



❖ RDBMS(SQL)

	ID	Name	Passion
Row 1 →	1	Happy	Football
Row 2 →	2	Amit	null

Users (Table)

❖ NoSQL

```
{
  "ID": 1,
  "Name": "Happy",
  "Passion": "Football"
}
```

Document 1

```
{
  "ID": 2,
  "Name": "Amit"
}
```

Document 2

Q. What is the difference between NoSQL & RDBMS? V. IMP.



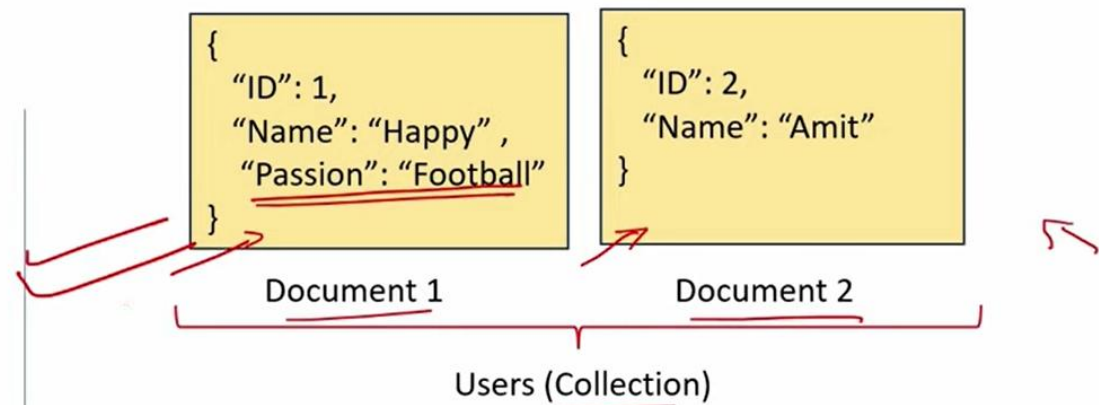
❖ RDBMS(SQL)

	ID	Name	Passion
Row 1	1	Happy	Football
Row 2	2	Amit	null

Users (Table)

1. RDBMS are designed to handle structured data (predefined and fixed schemas).
2. RDBMS use the relational model, where data is stored in tables with rows and columns.

❖ NoSQL



1. NoSQL databases are designed to handle structured, semi-structured, and unstructured data.
2. NoSQL databases use a variety of flexible data models, such as document, columnar, and graph.

Q. When to use RDBMS and when to use NoSQL DB in your applications?



Q. When to use RDBMS and when to use NoSQL DB in your applications?



1. RDBMS are suitable for applications that require complex transactions and data integrity(ACID), such as banking, finance, and e-commerce.



1. NoSQL databases are designed to handle large volumes of data with high-speed read and write operations, such as social media, IoT, and gaming.

